

The Wiring Is the Model

Coordinating, training, and streaming multi-agent systems
— a three-paper reader's guide

 Agentic Engineering Papers · catch-up digest, 5 June 2026

June 5, 2026

Chapter 0: The Wiring Is the Model

For a decade, the way to get a smarter system was to get a smarter model. That trade is mostly over. A strong single model is now a commodity you rent by the token, and the three papers in this bundle have all quietly stopped fighting that battle. They take the model as given and ask a different question: once every agent in your system is roughly as capable as it's going to get, what's left to tune? Their shared answer is the wiring — how the agents are decomposed, how their gradients are routed, how and when they talk. Coordination structure, not raw capability, is the variable they're all chasing.

What makes these three worth reading together is that they attack the same system from three different angles. MACU goes after its **shape**. Multi-Agent RL goes after its **training**. StreamMA goes after its **communication**. Read in sequence, they assemble into a single claim: the structure around the models is now as decisive as the models themselves.

Start with shape. MACU's whole result is that a manager drawing a live dependency graph of parallel sub-agents beats a stronger single agent doing the same work serially — and, the part to underline, it *composes* with backbone upgrades instead of competing with them. Swap in a better worker and both the single-agent and the multi-agent numbers rise together. That's the cleanest statement in the

bundle that coordination is a scaling axis of its own, one you can climb in parallel with model quality rather than instead of it.

Then training. The Multi-Agent RL paper takes the obvious next step — actually train those roles to cooperate — and finds there's no universal rule for how. Whether roles should share weights or stay isolated isn't a safety default you set once; it's governed by the workflow's topology. Same-role parallelism amplifies gradients into a late collapse under isolated policies; one dominant role quietly captures the shared weights under shared ones. The structure of the org chart determines how the training itself succeeds or breaks. Coordination doesn't just shape inference; it shapes whether learning converges at all.

Then communication. StreamMA's lever is timing. Stream each reasoning step downstream the instant it's produced instead of waiting for a full handoff, and you get both speed and — the counterintuitive part — *better answers*. Less context, delivered earlier, beats more context delivered late. And it surfaces a genuinely new knob: per-agent step count, a dimension you can scale at a fixed head count, orthogonal to adding more agents.

Watch for the thread that ties all three together, because it's the reason any of this pays off: agent work is **partially observable and non-uniformly reliable**, and that's exactly what makes structure matter. MACU has to persist what a later agent can't re-observe — a confirmation number, a closed tab, a file from a finished subtask — because a live OS won't let you look back. StreamMA wins precisely because early reasoning steps are trustworthy and late ones rot; corrupt the head instead of the tail and its advantage flips from +24 points to -36. Multi-Agent RL's collapses hide in exactly the per-role behavior the aggregate dashboards never show. In a fully observable, uniformly reliable world, you wouldn't need careful wiring. In the messy real one, the wiring is where the leverage lives — and every gain in these papers comes from respecting that mess rather than wishing it away.

So read the chapters as three cuts through one system. MACU shows you how to give it a shape that re-plans as evidence arrives. Multi-Agent RL shows you what happens when you try to train that shape, and where it quietly breaks. StreamMA shows you that even the timing of a single handoff is a design variable you've been leaving on the table. By the last page the question stops being "which model?" and becomes "how is it wired?"

Chapter 1: One Manager, Many Hands

Paper: [Multi-Agent Computer Use](#) · arXiv 2606.01533 · **Project page:** jykoh.com/multi-agent-computer-use

Picture a single agent planning a five-day Iceland camper-van trip through a browser. It reads a campground site, then a maps page, then hunts for a travel guide, and somewhere around action two hundred it loses the thread. The booking page from twenty steps ago is gone, the tab is closed, and it has no way to look back. This is how nearly every computer-use agent ships today: one model, one screen, one long serial march. It works for “change GIMP’s theme from dark to light.” It falls apart on anything that takes real planning.

The argument here is that the bottleneck isn’t the model — it’s the architecture. Computer-use agents (**CUAs**) are deployed as single serial actors, exactly the wrong shape for long-horizon work that wants decomposition, parallelism, and constant re-planning. The authors’ answer, **MACU** (Multi-Agent Computer Use), keeps whatever CUA you already have and wraps it in coordination. It beats strong single-agent baselines by **3.4 to 25.5%** across four desktop and web benchmarks, scales better as you spend more compute, and on long tasks finishes roughly **1.5x faster** in wall-clock time. The headline worth sitting with: coordination, not a stronger backbone, is a real scaling axis.

A manager that draws the graph and keeps redrawing it

MACU uses exactly two roles. A **manager** model reads the task and decomposes it into a **directed acyclic graph (DAG)** of subtasks — nodes are units of work, edges are dependencies. The manager then dispatches identical CUA subagents to run, in parallel, every node whose dependencies are already satisfied (the “ready frontier”). Each subagent runs a standard ReAct loop on its own isolated virtual machine, and MACU treats that loop as a black box. There’s no specialized vision module, no separate code agent, no expert hierarchy. Every worker is the same backbone; the only specialization comes from the instructions the manager writes.

What makes this more than a glorified task queue is that the graph is alive. Every time a subagent finishes, the manager gets its last few screenshots and its response, and is asked to revise: add a node, cancel one, rewrite an instruction, rewire a dependency. The initial plan is explicitly framed as a best guess — a hypothesis to be corrected once real observations come back. On the travel-planning example, that means launching hotel-review and flight-search branches at once, then spinning up alternate routes when the first ones get blocked, and finally fusing the surviving evidence into one answer.

Taking partial observability seriously

The part that separates computer use from the multi-agent work that came before it — the coding swarms, the math solvers, the science agents — is that those mostly operate in fully observable worlds. A live operating system is not that. Cookies expire, tabs close, background processes change state, and the screen relevant to a downstream subtask often cannot be re-observed once the subtask that saw it has ended. MACU makes this a first-class concern. Information a later agent won't be able to re-fetch is captured and carried forward through the manager and the DAG itself, including a file-management step: the manager inspects each subagent's filesystem diff, decides which files to keep, and hands them off — a Word doc from step one becoming the input step two uses to populate PowerPoint slides.

The numbers say coordination is its own lever

Across benchmarks the pattern is consistent. With an Opus 4.6 manager and a Qwen3.6-27B worker, MACU lifts OSWorld desktop success from 43.8% to 48.5% while cutting median time from 26.6 to 21.4 minutes. On Online-Mind2Web it goes 52.2% to 55.6%. The gains get dramatic where tasks are genuinely decomposable: WebTailBench-v2 climbs 20.8% to 29.5%, and on **Odysseys**, the long-horizon benchmark, success jumps from **8.5% to 34.0%** — and partial-credit rubric scores from 42.1% to 62.3% — while median time drops from 162 to 110 minutes, the 1.47x speedup.

Three ablations matter more than the topline. First, replanning is the engine: a planning budget of zero gets 25.0%, and an untouchable initial plan barely helps (27.8%) — but letting the manager keep editing the graph pushes it to 47.2% at budget 5 and 58.3% at budget 10. Second, MACU beats pass@8 up to 200 actions even though pass@8 cheats by checking the ground-truth evaluator, so decomposition and error correction, not just more tries, are doing the work. Third, and most useful for a roadmap: swap in a stronger worker and both the single-agent and multi-agent numbers rise together. MACU composes with model upgrades rather than competing with them.

What you could actually try

If you're building anything long-horizon on top of a computer-use or web-navigation agent, the borrowable idea is to stop modeling the task as one linear transcript and start modeling it as a live dependency graph. Put a manager model in front of your existing CUA — no backbone change needed — and have it write an initial DAG, fan out parallel workers on the ready frontier, and re-plan every time a worker reports back. Give that re-planning a real budget; the data says one shot of planning is nearly worthless and the value is almost entirely in revising as evidence arrives. And treat anything you can't re-observe — a confirmation number, a price you scrolled past, a file a subtask produced — as something you must explicitly persist and pass forward, not something a later agent can go look up. Because the workers are an interchangeable black box, the apparatus inherits every future CUA improvement for free — cheap to prototype now, safe to keep.

The authors close by noting what they didn't do: every gain here comes from coordination alone, with the manager and workers never trained for the job. The obvious next question is whether you can do better by actually training the agents to cooperate — and that's where the next chapter goes, into a study of multi-agent reinforcement learning that jointly trains LLM roles and asks a sharper question: when a shared policy across agents beats keeping each agent's policy isolated, and when each of those choices quietly breaks.

Chapter 2: Training the Whole Crew at Once

Paper: [When Does Multi-Agent RL Improve LLM Workflows? Workflow, Scale, and Policy-Sharing Tradeoffs](#) · arXiv 2605.24202 · **Code:** github.com/XHMY/marl-llm-workflows

A multi-agent LLM workflow is a small org chart. One model drafts, another critiques. Three models vote, a fourth tallies. An orchestrator hands work to a pool of workers, then a synthesizer writes the final answer. Each “role” is just the same base model wearing a different prompt, and the whole pipeline exists to squeeze more accuracy out of inference than a single forward pass can. But the obvious next move — train that whole crew end to end with RL, rewarding only the final answer — turns out to be a minefield almost nobody had mapped. This paper is the map.

The headline you can act on immediately: jointly training a multi-agent workflow with RL **usually does help** over the base model. Across a grid of three workflows, two tasks, and three model sizes, most cells came out ahead. But the size and even the *direction* of the gain depend on the workflow, the task, and the scale *together* — not on the one knob everyone reaches for first, which is whether the roles share weights.

The grid, and why it’s the whole point

The authors ran every cell of a controlled matrix. Three workflow topologies: **Eval-Opt** (a generator drafts, an evaluator critiques and may force a revision), **Voting** (three generators answer independently, one aggregator picks), and **Orch-Workers** (an orchestrator plans, three workers solve, a synthesizer combines). Two tasks, math and code. Three Qwen3 scales: 0.6B, 1.7B, 4B. And the axis the paper is really interrogating — **Shared-Policy** (all roles update one set of parameters) versus **Isolated-Policy** (each role type gets its own).

Crucially, every cell is measured against two controls: the untrained base model, and a single-agent RL run at the same scale and task. That second control is what lets them separate “RL helped” from “the *multi-agent structure* helped” — the discipline most ablations skip, and why the conclusions hold weight.

Higher ceiling, taller cliff

Isolated-Policy looks like the winner if you only glance at peak accuracy. Give each role its own parameters and the system climbs fast and reaches a higher ceiling — at 1.7B on math, Eval-Opt under Isolated-Policy hit 60% versus Shared-Policy’s 50.5%, a clean ten-point edge. The diagonal plot of matched cells leans Isolated’s way.

The catch shows up late in training. Isolated-Policy runs more often **fall off a terminal accuracy cliff** — validation climbs, peaks, then collapses in the back half of the run. Shared-Policy doesn’t do this

as dramatically; it plateaus earlier and lower but holds. So the real tradeoff isn't "better" versus "worse." It's **ceiling versus floor**: Isolated buys you a higher peak and a higher chance of watching it evaporate if you train too long.

And here's the trap that makes this paper worth reading twice: Shared-Policy does **not** eliminate failure. It *redistributes* it. The instability Isolated-Policy expresses as a late cliff, Shared-Policy expresses as a quieter pathology — sometimes one the usual training dashboards never show.

What's actually breaking, in plain terms

The paper traces both failure modes to the same root: outcome-reward gradients pile onto one role's parameters, and the policy-sharing choice merely decides what shape the pile takes.

Under **Isolated-Policy**, the culprit is same-role parallelism. When a workflow has several copies of one role answering the same prompt — the three generators in Voting, the three workers in Orch-Workers — that role collects several gradient samples per step that all point the same way (same problem, same advantage sign). The updates reinforce each other, the role drifts faster than the base model can absorb, and it overshoots into collapse. In Voting at 1.7B math, the generator's policy-ratio diagnostic climbs roughly 30x over training while the aggregator's stays flat — and validation peaks and slides right alongside the generator's drift.

Under **Shared-Policy**, the failure is **role capture**. One shared policy serves every role, so whichever role contributes the most distinctive gradient mass quietly pulls the shared weights toward its own idiom. The evaluator slot, asked to emit a terse verdict, starts writing full code blocks on code tasks or long re-derivations on math. In Voting at 4B math, the aggregator — designed to stamp out a one-line selection — drifts into verbose justification, *while the voter-side token metrics sit unchanged at their starting values*. The collapse is at the answer-correctness layer, invisible to token-level training stats. The slot tasked with scoring the work starts producing the work.

What a reader could try

If you're RL-training a multi-agent workflow — or thinking about it — the operational lessons are concrete:

- **Don't treat shared parameters as a free stability win.** That's the instinct ("fewer params, less to go wrong") and it's wrong. Sharing reroutes the training pressure; it doesn't drain it. Pick sharing based on your *topology*, not on a vague sense that it's safer.
- **If you go Isolated, watch for the late cliff.** The danger isn't early — it's the back half. Checkpoint often, track validation past the peak, and be ready to early-stop. Same-role parallelism (multiple voters, multiple workers) is the amplifier, so workflows with parallel slots warrant the closest watch.

- **If you go Shared, monitor per-role *behavior*, not just aggregate accuracy and token-level metrics.** Role capture hides from gradient norms and entropy. The tell is on the trajectory side: is one role's output length, format, or style migrating toward another's over training? Log per-role output shape and diff it against the early checkpoints. When one role dominates the per-step gradient mass, that's your capture risk lighting up — aggregate metrics stay calm right until the answer layer breaks.

The deeper reframe is the part to internalize: policy-sharing is a **design choice with workflow- and task-conditional tradeoffs**, not a default safety setting. There's no universally stable answer to "share or isolate?" — there's only the answer for *your* topology, *your* task, and how long you train.

That framing — that where and how information flows between agents reshapes the whole system's behavior — sets up the next chapter nicely. There, the lever isn't weight-sharing but timing: StreamMA streams each reasoning step to downstream agents the instant it's generated, rather than waiting for a role to finish. The surprise is that piping partial thoughts forward doesn't just cut latency — it improves the final answers, too.

Chapter 3: Agents That Talk Before They Finish Thinking

Paper: [Streaming Communication in Multi-Agent Reasoning](#) · arXiv 2606.05158 · **Code:** github.com/EnVision-Research/StreamMA

Every multi-agent system you’ve ever built has an unspoken rule: an agent finishes its whole answer before passing anything to the next agent. It feels obviously correct — you wouldn’t hand someone half a thought. But that politeness has a price, and StreamMA argues the price is paid twice: once in latency, and once, surprisingly, in accuracy.

The fix is almost embarrassingly simple. Instead of waiting for the full response, stream each **reasoning step** to the next agent the moment it’s produced. The downstream agent starts on step one while the upstream agent is still generating step two. Adjacent agents pipeline. And across eight benchmarks, two frontier models, and three topologies, this faster setup is also the more accurate one.

The counterintuitive part: less context, better answers

Here’s the result that should make you pause. Conventional wisdom says more information yields better decisions — give the downstream agent the complete response and it has the most to work with. StreamMA finds the opposite holds for multi-step reasoning. Working from the *partial* early output beats working from the full output.

The mechanism is **non-uniform reasoning quality**. On hard problems, an LLM’s early steps tend to be reliable, and later steps degrade as error accumulates. The old “generate-then-transfer” protocol forces the downstream agent to swallow the entire chain, clean early steps mixed with error-prone late ones. Streaming lets it begin from the trustworthy prefix. By the time the degraded tail steps arrive, the downstream agent has already formed its own independent reasoning trajectory, so the bad steps get diluted instead of swallowed whole. When context arrives matters more than how much arrives.

The paper nails this down with a controlled perturbation. Take an upstream trajectory and corrupt only its *tail* steps: streaming stays nearly intact and leads the serial baseline by **+24.0 pp**. Now corrupt only the *head* steps instead: the gap flips to **−36.0 pp**. The asymmetry is the whole story. Streaming wins precisely when early reasoning is solid and late reasoning rots — which, on real math/science/code problems, is the common case.

The numbers

Across AIME 2025/26, HMMT 2026, GPQA-Diamond, HLE, and three LiveCodeBench tasks — with Chain, Tree, and Graph topologies — StreamMA beats the serial baseline in every single average cell. On Claude Opus 4.6-high it lifts accuracy by an average **+7.3 pp**, peaking at **+22.4 pp** on HMMT 2026. On GPT-5.4-medium the gains are smaller but still consistent (+1.5 pp over serial, +4.9 pp over the single-agent baseline).

One pattern worth internalizing: the gain scales inversely with how good the serial baseline already was. Where serial was strong (Tree on Claude, ~79%), StreamMA added +3.4 pp. Where serial was weaker (Chain and Graph, ~73%), it added +8.2 and +10.4. The intuition fits the mechanism — a strong serial run has few step-level errors to neutralize, a weak one has many. Near a benchmark ceiling (GPT on LCB-T at ~99%), the cells barely move.

Crucially, this isn't prompt engineering in disguise. StreamMA inherits the serial baseline's prompts, decoding settings, and runtime verbatim, adding only a one-line `END_STEP` boundary marker to each downstream agent's system prompt. The transmission granularity is the only thing that changes.

A second scaling knob you didn't know you had

The deeper contribution is a new dimension to scale. Everyone knows you can add more agents. StreamMA shows that at a *fixed* agent count, increasing the number of reasoning steps each agent produces — call it **step-level scaling** — improves both accuracy and speedup, and it's fully composable with agent-count scaling.

The HMMT 2026 sweep makes it concrete. Scaling agents alone from 2 to 64 lifts accuracy from 58.3% to 68.2%. Then, holding 64 agents fixed and pushing per-agent steps to 64, accuracy climbs further to 73.5% — a +5.3 pp gain orthogonal to anything agent-count scaling gave you — while wall-clock speedup hits 26.9× (83% of the theoretical bound). One nuance: models don't do this on their own. Left to "auto," an LLM defaults to coarse, few-step reasoning; the large-step regime has to be explicitly unlocked through prompting.

The paper backs all of this with the first closed-form joint analysis of the stream, serial, and single protocols — an effectiveness ordering across six regimes, a speedup upper bound, and an exact cost ratio. The honest version of the ordering: streaming is *strictly* best only when agents are head-strong and tail-weak. In other regimes (self-correcting agents that fix early mistakes late, or problems so hard every step is harmful), serial or single can match or beat it. That's not a weakness — it turns the theory into a protocol selector. Profile your task's step-correctness shape, and the theorem tells you which protocol to run.

What you could try

If you run any pipeline where one model's output feeds another — a planner into a coder, a retriever into a synthesizer, a drafter into a critic — stop waiting for full completions. **Pipeline your stages by streaming partial output** so the downstream stage starts on the first chunk while the upstream is still generating. Most agent frameworks already expose streaming; the missing piece is wiring the stream into the *next agent's input* instead of just rendering it to a screen.

Second, **lean on the early steps**. If your downstream agent only needs to know the upstream's direction, not its final committed answer, feeding it the reliable prefix may beat feeding it the whole chain — especially on hard problems where the tail is where things go wrong. Test this directly: run a downstream stage on the upstream's first few steps versus its full output and compare.

Third, treat **per-agent step count as a tunable knob**, not a fixed property of the model. Prompt for more, finer reasoning steps and watch both accuracy and throughput. It's a lever you can pull independently of how many agents you're paying for.

These three papers circled the same realization from different angles: a multi-agent system is not just a pile of models, it's a communication structure, and the structure is where the leverage lives. The earlier chapters showed that coordination and context-passing decide whether agents amplify or corrupt each other. StreamMA closes the loop by showing that the *timing and granularity* of that passing — not just the topology or the head count — is a first-class design variable. How agents talk to each other turns out to be as real a lever as how many of them there are, and we've barely started pulling it.